A PIPELINE FOR INTERPRETABLE NEURAL LATENT DISCOVERY

Anonymous authors
Paper under double-blind review

ABSTRACT

Mechanistic understanding of the brain requires interpretability of large-scale neuronal computations. Many latent variable model approaches applied to such datasets excel at decoding but yield opaque latent spaces. We address this with NLDisco, a pipeline for interpretable neural latent discovery. Motivated by successful applications of sparse dictionary learning in AI mechanistic interpretability, NLDisco encourages hidden layer neurons in sparse encoder-decoder models to learn interpretable structure in dense neural activity. A flexible, open-source, user-friendly software package supports interpretable latent discovery across recording modalities and experimental paradigms. We validate the pipeline on both synthetic and real extracellular recordings across species and brain regions, showing that it recovers ground-truth features in synthetic data and reveals meaningful representations in real data. These results highlight the pipelines potential to advance neuroscientific discovery and provide a foundation for future methodological extensions and applications in interpretable modeling of neural activity.

1 INTRODUCTION

Advanced extracellular recording technologies now enable neuroscientists to capture unprecedented volumes of high-dimensional neural[1] data at single-cell resolution (Steinmetz et al., 2021; Raducanu et al., 2017; Cai et al., 2016; Villette et al., 2019; Ouzounov et al., 2017; Ahrens et al., 2013). Understanding the computational principles driving neural activity requires extracting interpretable features from this data. Traditional dimensionality reduction techniques, while useful for visualization and basic analyses, often make invalid assumptions about or altogether fail to disentangle distributed representations, and as a result typically cannot provide a mechanistic understanding of the underlying computations (Cunningham & Yu, 2014; Humphries, 2021). Recent work has produced promising latent variable model (LVM) approaches capable of identifying low-dimensional subspaces that can accurately decode aspects of behavior and environment (Song et al., 2025; Schneider et al., 2023; Le & Shlizerman, 2022; Keshtkaran & Pandarinath, 2022; Yu et al., 2009; Macke et al., 2011; Gao et al., 2016; Low et al., 2018; Jensen et al., 2020; Hernandez et al., 2020; Kim et al., 2021; Hurwitz et al., 2021; Schimel et al., 2022; Kudryashova et al., 2023; Ye et al., 2023; Gondur et al., 2024; Pellegrino et al., 2024; Sani et al., 2024; Pals et al., 2024; Zhang et al., 2024; George et al., 2025; Perkins et al., 2025; Schmutz et al., 2025). However, these methods typically value decoding accuracy over latent interpretability, and consequently all have at least one of the following limitations: poorly interpretable latents, lossy mapping to and/or poor reconstruction of neural data from latent space, priors on the latent and/or neural data space, requirements of additional behavioral, environmental, and/or trial-structured neural data, supralinear scaling with respect to dataset size, or high implementational complexity.

We address each of these limitations in NLDisco, which provides a simple, user-friendly library for interpretable latent discovery from high-dimensional neural data. We consider a latent's interpretability in two key aspects: 1) its correspondence to a specific external variable – a "natural"

[1]By default, "neural" and "neuron" will refer to neurobiology; e.g. we call biological neuronal spike data "neural activity". In contrast, activity and neurons from artificial neural networks will be explicitly prefaced; e.g. "artificial neural activity" or "the model's neurons".

behavioral or environmental feature[2]; 2) its explicit composition from contributing neural activity. Unlike other approaches that require a search for meaningful directions or dynamics in a latent space, NLDisco outputs individual latents that can be directly assessed for interpretability. Additionally, while we showcase NLDisco on spike data, it can be readily used with virtually any other neural recording modality as the only data requirement is any predefined spatiotemporal binning.

NLDisco trains shallow, overcomplete, sparse encoder-decoder (SED) neural network models (Olshausen & Field, 1997; Vincent et al., 2008) whose individual dictionary elements – hidden layer neurons – ideally represent interpretable latents. This particular LVM approach has had many successes in the field of AI mechanistic interpretability, in which such models have typically been implemented as sparse autoencoders, transcoders, or crosscoders (Lindsey et al., 2024; Cunningham et al., 2023; Bricken et al., 2023; Templeton et al., 2024; Dunefsky et al., 2024; Ameisen et al., 2025; Lindsey et al., 2025). For neural data, SEDs are additionally motivated by the principle that we need not explicitly impose structure on the latent space nor model its dynamics to discover interpretable variables.

Specifically, NLDisco bypasses the need to enforce non-Markovian dynamics, which are often a property of an incomplete observation of a system rather than the system itself; any dynamical system can be represented as Markovian via sufficient state augmentation (Takens, 1981). Given this, even methods that appear non-Markovian can be seen as attempts to approximate a complete underlying state. For example, the forward dynamics of a recurrent neural network (RNN) are Markovian in its hidden state (Sussillo & Barak, 2013; Goodfellow et al., 2016), and generalized linear models (GLMs) with history filters (Pillow et al., 2008; Truccolo et al., 2005) use past activity to augment the present state. This perspective aligns with several findings in neuroscience, from activity in motor cortex reflecting a low-dimensional, implicitly Markovian dynamical system (Churchland et al., 2012; Shenoy et al., 2013), to predictive-coding formulations that posit that the brain maintains an internal state sufficient to predict sensory streams, i.e., a Markovian latent process (Rao & Ballard, 1999; Doya et al., 2007; Friston, 2010). Grounded in this perspective, NLDisco forgoes the challenge of modeling intricate dynamics and instead directly extracts meaningful constituent features of the underlying neural state (Figure S1).

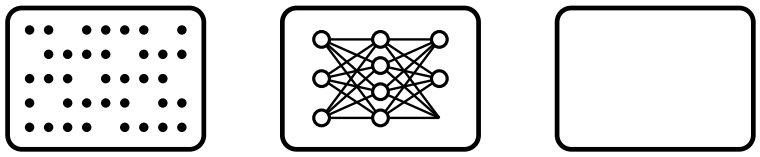

Figure 1: The NLDisco pipeline.
The NLDisco pipeline has 4 stages: 1) Spatiotemporal binning and processing of neural data; 2) Training a model; 3) Evaluating the model; 4) Evaluating the model's latents for feature interpretability. Steps 1-3 can be either semi- or fully-automated.

2 METHODS: THE NLDISCO PIPELINE

The NLDisco pipeline transforms high-dimensional neural data into a set of interpretable latents in four stages (Figure 1), the first three of which can be fully automated.

2.1 DATA PREPROCESSING

The pipeline's first stage prepares neural data for model training. NLDisco includes utilities to process outputs from common spikesorters (e.g. Kilosort (Pachitariu et al., 2016)) by aggregating, binning, and normalizing spike times into a 2D matrix of time by space, in which each bin contains neural activity from a neural unit for a specific time period. Normalization operations include min-max and z-scoring, applied across either the spatial or temporal dimension. This processing is readily adaptable to other forms of acquired neural data, such as ROIs from calcium imaging data processed

[2]A "feature" is a sufficiently interpretable latent. We illustrate this in Results.

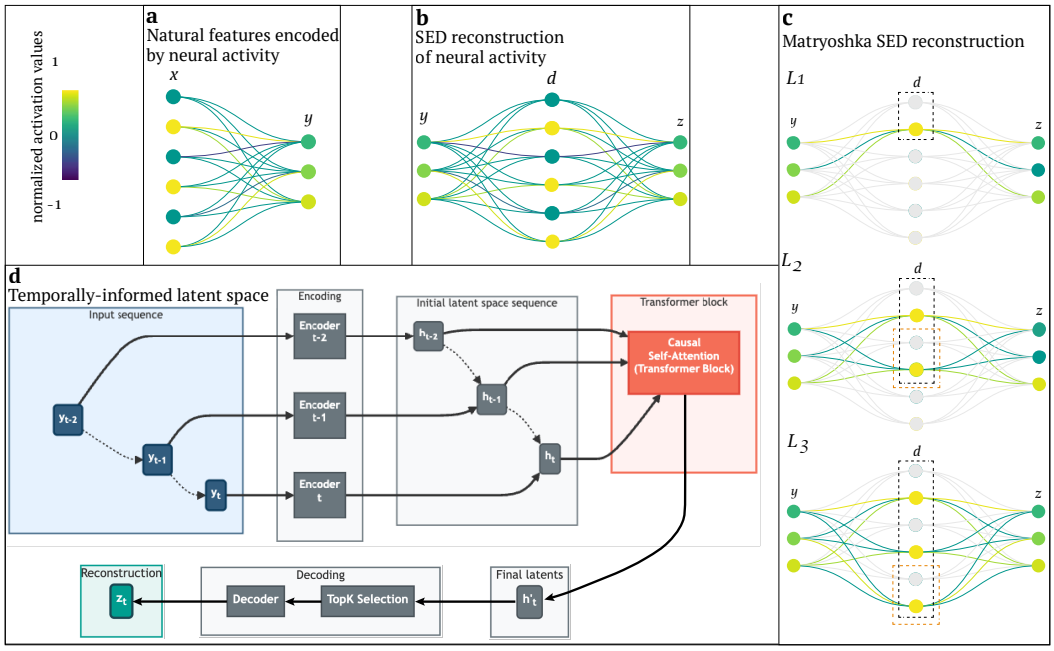

Figure 2: Model architecture considerations
(**a**) Natural, "real-world" features x are encoded by neural activity y. In this example, three active features are simultaneously represented by the joint activity of three neurons. (**b**) A SED reconstructs neural activity z based on y via sparse dictionary elements d. When training is successful, d corresponds to x: sparse dictionary elements (i.e. model neurons) represent natural features. If z tries to recreate y exactly ($\hat{y}$), the model is an autoencoder; in other scenarios (e.g. z is separate but dependent on or related to y) it is a transcoder. (**c**) A Matryoshka SED segments the latent space into multiple nested levels, each of which attempts to do a full reconstruction of the target neural activity. The black boxes indicate the latents involved in a single level, while the light-red boxes indicate the additional latents used at lower-levels. In this example, k=1 for top-k selection of latents to recruit for reconstruction at each level (the yellow neuron within each light-red box). Latents in the highest-level (L_1) will often correspond to high-level features (e.g. a round object), while latents exclusive to the lowest-level (L_3) will often correspond to low-level features (e.g. a basketball). (**d**) Incorporation of a transformer block with sequence input allows imbuing the latents with temporal information corresponding to the evolution of the sequence, which can lead to improved reconstruction and interpretability. Each sample in the input sequence is transformed by the same encoding dictionary matrix in parallel to yield a latent space sequence. Causal self-attention is then performed on this sequence of latents via a single, small multi-head transformer block, yielding a final latent space. The latents are sparsified via top-k and transformed by the decoding dictionary matrix to yield the neural data reconstruction, as in the single, non-sequential input case.

by tools like Suite2p (Pachitariu et al., 2017). Alternatively, users may pass their own preprocessed neural data directly to the model training stage.

2.2 Model Training

In stage 2, a SED model is trained to reconstruct the processed neural data from latent, sparsely active dictionary elements – the model's hidden layer neurons[3]. The end goal is for these latents to represent disentangled, interpretable representations[4] encoded by the neural activity (Figure 2a,b). Sparsity encourages the model to discover a monosemantic dictionary, in which each latent corresponds to a single feature. For instance, a latent from a model trained on visual cortex activity may correspond to a particular property of a visual stimulus, while a latent from a model trained on motor cortex activity may correspond to a specific aspect of movement.

The model can be configured as an autoencoder (Cunningham et al., 2023) to reconstruct its own input (e.g. V1 activity based on V1 activity), or as a transcoder (Dunefsky et al., 2024) to reconstruct a dependent or related target of its input (e.g. V2 activity based on V1 activity, or V1 activity on day

[3]In a SED model: model neuron = dictionary element = latent
[4]In our vernacular: interpretable latent ≈ feature ≈ representation

2 based on V1 activity on day 1). These configurations allow for examining the intrinsic representational structure of a neural population and how representations may be transformed across brain regions and differ over time or between subjects. In each case, the model is trained to minimize the difference between the reconstructed and target neural data. The full training procedure is summarized in Model training procedure. Here we discuss three key components: the batch top-k sparsity, Matryoshka architecture, and optional integration over latent space history via self-attention.

A batch top-k operator preserves only the $k \cdot B$ latents with the highest activation magnitudes across a batch of size B (Bussmann et al., 2024). In contrast to indirect approaches (e.g. an L1 penalty), batch top-k directly controls sparsity while flexibly permitting the number of active latents to vary per sample in the batch, aligning with the variable complexity of neural states. A novel variant of the Matryoshka SED architecture (Bussmann et al., 2025) segments the latent space into nested hierarchical levels, where each level attempts a full reconstruction of the target neural activity (Figure 2c). This allows for multi-scale feature representation at single timepoints, and mitigates "feature absorption," a common issue where a more specific feature subsumes a portion of a more general feature (Chanin et al., 2025). For example, a model can learn that a "drifting grating" is a subset of the more general "grating" visual stimulus class without sacrificing the representation of either (Mouse spike data in a passive visual task). Lastly, if input data is provided as a sequence of timebins rather than a single timebin, an optional transformer block can integrate information over the latent space history via self-attention (Figure 2d), enabling the model to learn features corresponding to evolving temporal dynamics, rather than just instantaneous neural patterns. This optional mechanism can be viewed as a form of explicit state augmentation; while the underlying Markovian state can theoretically be inferred from a single timebin, providing a sequence history can serve as a powerful inductive bias, making it easier for the model to discover features that unfold over longer timescales.

Effectively searching the large and flexible model space created by these architectural options is crucial, as we empirically find that these components can significantly improve both reconstruction accuracy and feature interpretability (Model architecture). To streamline this search with minimal manual specification, NLDisco supports fully automated training via hyperparameter sweep configurations that support local or distributed training across one or multiple CPUs or GPUs (Hyperparameter sweeps).

2.3 Model evaluation

Folowing training, model quality is evaluated using a suite of quantitative metrics. NLDisco adapts domain-general core metrics from the established mechanistic interpretability benchmark SAEBench (Karvonen et al., 2025), and introduces others tailored for neural data. This stage serves as a critical checkpoint: while these metrics do not directly assess latent interpretability, they allow a researcher to determine if the model has learned a sparse, accurate, robust representation of the neural data. A model that performs well across these metrics is a strong candidate for the subsequent, final pipeline stage of evaluating individual latents.

Model evaluation centers on the trade-off between reconstruction fidelity and latent dictionary sparsity. High reconstruction fidelity is trivial to achieve with a large and dense code but undermines the goal of finding disentangled, interpretable latents. NLDisco therefore first reviews dictionary quality via latent sparsity metrics (Figure S2 top). The mean L0 norm measures the average number of active latents per sample, set indirectly by the batch top-k parameter, while the latent activity density visualizes the firing distribution across all latents. Together these metrics reveal whether the model has learned an efficient code that uses many latents intermittently while avoiding common pitfalls such as "representational collapse" into too many constantly active latents and/or widespread "feature death", in which a large portion of the dictionary never activates. These sparsity metrics are complemented by a couple of reconstruction fidelity metrics: variance explained (R^2) and cosine similarity of reconstruction-to-target neural activity across both spatial and temporal dimensions (Figure S2 bottom). High scores in these metrics confirm that the sparse code is a sufficient representation that has captured salient information present in the target neural data.

For a deeper diagnosis, several optional metrics are available (**??**). To check whether explanatory power is well-distributed, a variance attribution analysis quantifies the overall reconstruction variance explained by each individual latent. Additionally, to check if temporal information present

in the target neural data is preserved by the latent reconstruction, a spectral frequency analysis compares the reconstruction's frequency content to that of the target neural data, given a specified duration.

Overall, these metrics provide a high-level overview of model quality, and to facilitate bespoke evaluation, the model's raw parameter, gradient, and activation data during training and evaluation are accessible.

2.4 LATENTS EVALUATION

Lastly in stage 4, the latents produced by the model are evaluated for interpretability. This can include visualizing their activation patterns over time and experimental conditions, as well as assessing their decoding performance. We detail typical approaches for exploring latents in Results.

3 RESULTS

- We evaluate NLDisco on one synthetic single-unit dataset with ground-truth latents, and three real extracellular electrophysiology, open-source, high-yield, single-unit datasets, across multiple, distinct brain regions in multiple species. (See **??** for additional dataset details.)

- Overall, we show that NLDisco can:

1. Discover discrete and continuous, environmental and behavioral features
2. Robustly find same features across time and subjects.
3. Find features across different timescales (i.e. that persevere for different durations)
4. Find hierarchical features.

- On the synthetic dataset, we show that NLDisco can recover all ground-truth latents, and use them to decode natural features of interest.

- On one natural dataset, we compare NLDisco to other popular approaches that have also been applied to the same dataset, and show that NLDisco more clearly reveals certain features than these other methods, while maintaining high reconstruction and decoding accuracy.

- On the other two natural datasets, we show that NLDisco can reveal expected and unexpected features (optinally / time-permitting, that CEBRA and LangevinFlow don't find), highlighting its use as a powerful tool for neuroscientific discovery.

—

3.1 SIMULATED RAT SPIKE DATA IN A NAVIGATION TASK

We simulated the activity of hippocampal neurons during a navigation task in a virtual linear environment. The virtual rat moved along a 1-m linear track (Figure 3), with its velocity sampled from an Ornstein-Uhlenbeck (OU) process:

$$dv_t = \theta\,(\mu - v_t)\,dt + \sigma\,dW_t$$

with the following parameters: θ = 1.0 (mean reversion), μ = 0.0 (long-run mean velocity, m/s), σ = 0.4 (volatility), v_0=0.0 (initial velocity). The full simulation lasted 10 minutes (600 s) at a temporal resolution of 100 Hz.

We modeled four place cells along the track. Two cells had relatively broad place fields (Gaussian width σ = 20 cm), while the other two had narrower place fields (σ = 10 cm). This heterogeneity was intended to approximate recordings from different hippocampal subregions along the dorsoventral axis. Place fields were modeled as Gaussian functions of position with equal firing probability regardless of running direction.

The two broad fields were centered at 200 cm and 800 cm, while the two narrower fields were embedded within them at 300 cm and 900 cm, respectively. Baseline firing rates were fixed at 0.1 Hz, and peak firing rates were set to 20 Hz, 16 Hz, 18 Hz, and 15 Hz from left to right along the track (0-1000 cm) (Figure 4).

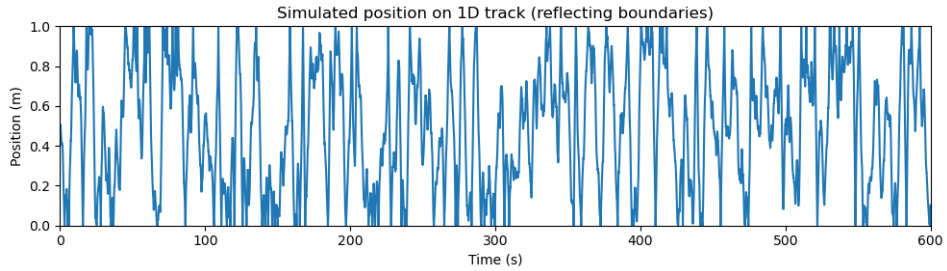

Figure 3: Simulated movement along the track with the Ornstein-Uhlenbeck formula. The simulated rat sampled all the positions along the track multiple times over 600 seconds.

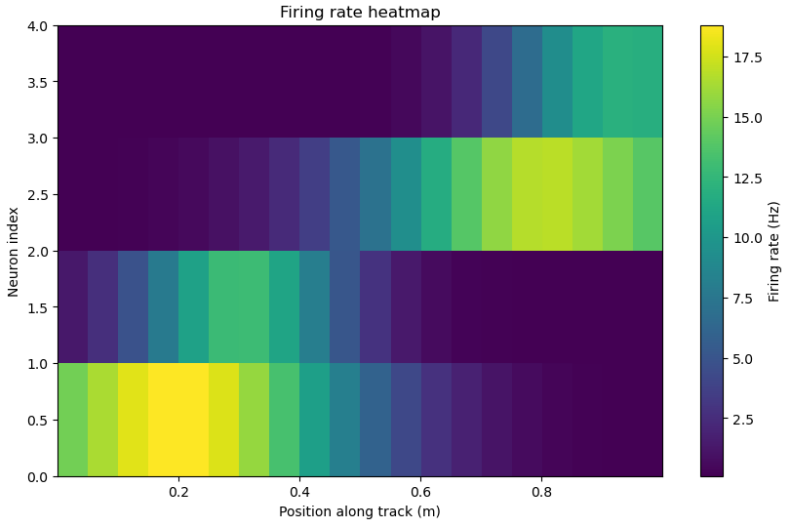

Figure 4: Ratemap of 4 simulated place cells in a 1-meter linear track. Their place field cover the whole length of the track.

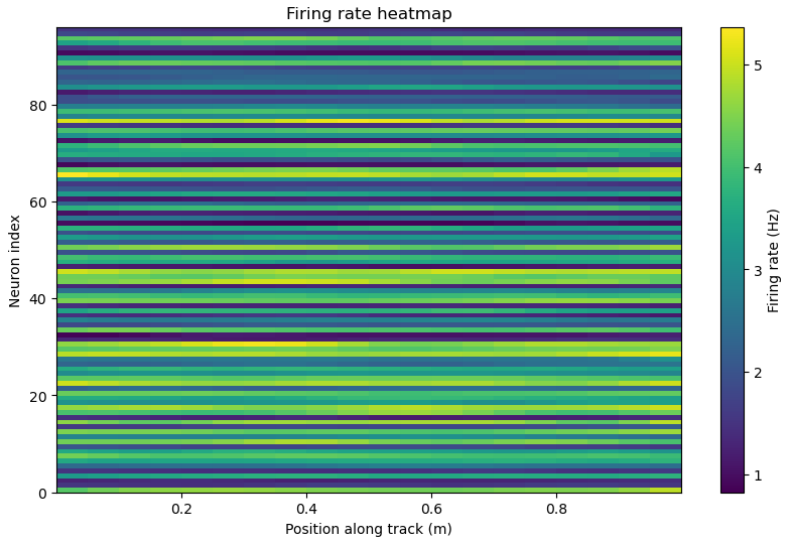

Figure 5: Simulated noise along the track, the neurons had a firing rate between 1 and 5 Hz, and their firing was independent of position.

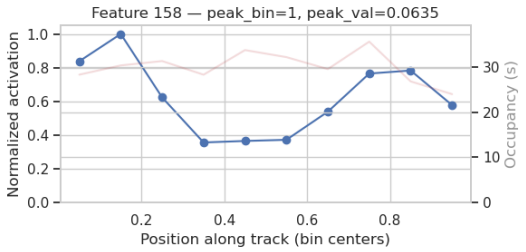

Figure 6: Enter Caption

To mimic a larger hippocampal population, we also simulated 96 noise neurons whose activity was independent of position. Their mean firing rates were sampled uniformly between 1 and 5 Hz and held constant across the entire recording. Spiking activity was generated as Bernoulli draws with rate $\times$ timestep probability (figure 5).

We extracted the spike count matrix from the simulation, with dimensions *time samples* $\times$ *neurons*, and used it to train a Matryoshka Sparse Autoencoder (SAE). The model was trained with the following hyperparameters: dsae_topk_map={64:1, 128:2, 256:4}, dsae_loss_x_map={64:1.0, 128:1.25, 256:1.5}.

The SAE successfully discovered a range of latent features. As expected, several features primarily reflected activity from the noise neurons, capturing random fluctuations without spatial structure. Crucially, however, a substantial subset of latent features encoded position along the track.

Two main types of spatial representations emerged:

1. **Global position features**: latents whose activation varied gradually along the track, suggesting a coarse encoding of absolute position.

2. **Local position features**: latents that exhibited sharp peaks at specific locations, resembling place-cell-like activity and corresponding to localized representations of position.

These results demonstrate that the Matryoshka SAE can disentangle structured spatial representations from unstructured noise, recovering both global and local spatial codes (see Fig. X).

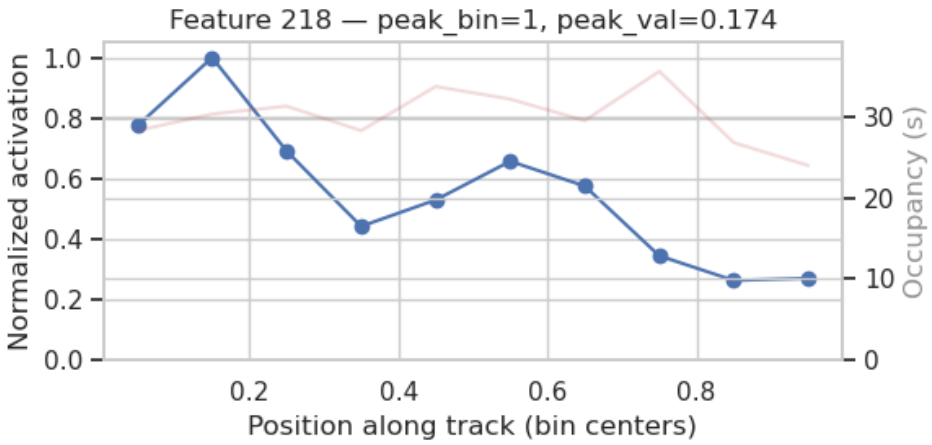

Figure 7: Enter Caption

3.2 Macaque spike data in a reaching task

Here we compare NLDisco to other popular methods that have also been applied to this dataset.

To do a fair comparison with NLDisco's main goal of unsupervised interpretable latent discovery, we detail comparisons (Methods Comparison) with paper-published methods that:

1. Claim an interpretable latent space among their primary goals (as opposed to e.g. only strong decoding performance)
2. Don't require multimodal nor trial-structured data

Among these, we visualize results from LangevinFlow and CEBRA, as they represent current neural latents benchmark[ref] state-of-the-art methods for an autoencoder approach and non autoencoder approach, respectively.

We also include sparseNMF[ref], t-SNE[ref], and UMAP[ref] as baselines, as they are widely used general methods for dimensionality reduction and latent extraction. (We don't show ICA, as (ICA < CEBRA) nor PCA, as (PCA < sparseNMF)).

Brief dataset info...

- Task info & metadata
- Brain regions & units

Features found and decoding accuracy comparisons with LangevinFlow and CEBRA...

1. Direction
2. Velocity
3. Movement onset
4. Trial type
5. Features across different timescales (i.e. that persevere for different durations)
6. Communication across brain regions??

3.3 Mouse spike data in an ethological foraging assay

- We analyze data from the Aeon project, which provides continuous long-term recordings of freely behaving mice in enriched environments. This dataset allows us to study neural dynamics during naturalistic behaviors over extended time periods, from minutes to weeks.

- Brief dataset info...

- Features found and decoding accuracy comparisons with LangevinFlow and CEBRA...

See Software and data availability for notebooks reproducing these results.

4 DISCUSSION

4.1 CONTRIBUTIONS

This paper makes the following key contributions:

1. We introduce a pipeline for applying sparse dictionary learning methods specifically designed to discover interpretable neurobiological latents ("features").
2. Our pipeline is end-to-end from spatiotemporally binned neural data to interpretable latent discovery: containns functionality for neural data preprocessing, model training, model evaluation, and feature evaluation.
3. We provide systematic evaluation on multiple synthetic and natural large-scale neurobiological datasets, demonstrating the effectiveness of our approach across different species, brain regions, recording techniques, experimental conditions, and spatiotemporal scales.
4. We show that MSAE-derived features enable effective decoding of behavioral and stimulus variables, validating their biological relevance.
5. We release our code open-source with interactive tutorials for facilitating broader adoption in the neuroscience community.

4.2 PROS AND CONS

Advantages:

- **Interpretability**: The sparse and multi-scale nature of the learned features facilitates direct interpretation and comparison with existing neuroscience knowledge.
- **Scalability**: The model architecture is designed to scale to large datasets with thousands of neurons and millions of time points.
- **Flexibility**: The framework can be adapted to various data modalities, including spike trains, calcium imaging, and LFP/EEG signals.

Limitations:

- Model performance is sensitive to the choice of hyperparameters, requiring careful tuning and validation.
- While our model reveals correlational structure, it does not inherently infer causal relationships between neural features and behavior.
- The model is a simplified representation of neural circuits and does not capture all aspects of biological complexity.
- Latent interpretability is subject to human subjectivity.

4.3 FUTURE WORK

- Actually apply to other recording modalities
  - LFP & EEG (reconstruct voltage values from C channels over T time)
  - Calcium imaging (reconstruct dF/F values from R RoIs over T time)
- Actually apply to multimodal data
  - e.g. reconstruct spikes from spikes + multimodal behavior (e.g. pose-tracking, gaze-tracking, etc.)
- Use history to improve reconstructions and feature interpretability?

  - While our codebase optionally allows for using history length, in practice we did not get improved reconstructions nor more interpretable latents, but in theory this approach could be improved to do so.
- CLTs & SCCs
  - Cross-region prediction
  - Brain diffing

REFERENCES

Misha B Ahrens, Michael B Orger, Drew N Robson, Jennifer M Li, and Philipp J Keller. Whole-brain functional imaging at cellular resolution using light-sheet microscopy. *Nature Methods*, 10 (5):413–420, 2013. doi: 10.1038/nmeth.2434. URL https://doi.org/10.1038/nmeth.2434.

Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/methods.html.

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nicholas L. Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/2023/monosemantic-features/index.html#appendix-feature-ablations.

Bart Bussmann, Patrick Leask, and Neel Nanda. Batchtopk sparse autoencoders, 2024. URL https://arxiv.org/abs/2412.06410.

Bart Bussmann, Noa Nabeshima, Adam Karvonen, and Neel Nanda. Learning multi-level features with matryoshka sparse autoencoders. *arXiv*, 2025. URL https://arxiv.org/abs/2503.17547.

Denise J Cai, Daniel Aharoni, Tristan Shuman, Justin Shobe, Jeremy Biane, Weilin Song, Brandon Wei, Michael Veshkini, Mimi La-Vu, Jerry Lou, Sergio E Flores, Isaac Kim, Yoshitake Sano, Miou Zhou, Karsten Baumgaertel, Ayal Lavi, Masakazu Kamata, Mark Tuszynski, Mark Mayford, Peyman Golshani, and Alcino J Silva. A shared neural ensemble links distinct contextual memories encoded close in time. *Nature*, 534(7605):115–118, 2016. doi: 10.1038/nature17955. URL https://pubmed.ncbi.nlm.nih.gov/27251287/.

David Chanin, James Wilken-Smith, Tomá Dulka, Hardik Bhatnagar, Satvik Golechha, and Joseph Bloom. A is for absorption: Studying feature splitting and absorption in sparse autoencoders. *arXiv*, 2025. doi: 10.48550/arXiv.2409.14507. URL https://arxiv.org/abs/2409.14507.

Mark M. Churchland, John P. Cunningham, Matthew T. Kaufman, Justin D. Foster, Paul Nuyujukian, Stephen I. Ryu, and Krishna V. Shenoy. Neural population dynamics during reaching. *Nature*, 487(7405):51–56, 2012. doi: 10.1038/nature11129. URL https://doi.org/10.1038/nature11129.

Pierre Comon. Independent component analysis, a new concept? *Signal Processing*, 36(3): 287–314, 1994. doi: 10.1016/0165-1684(94)90029-9. URL https://doi.org/10.1016/0165-1684(94)90029-9.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv*, 2023. doi: 10.48550/arXiv.2309.08600. URL https://arxiv.org/abs/2309.08600.

John P Cunningham and Byron M Yu. Dimensionality reduction for large-scale neural recordings. *Nature neuroscience*, 17(11):1500–1509, 2014.

Kenji Doya, Shin Ishii, Alexandre Pouget, and Rajesh P. N. Rao (eds.). *Bayesian Brain: Probabilistic Approaches to Neural Coding*. MIT Press, Cambridge, MA, 2007. URL https://mitpress.mit.edu/9780262516013/bayesian-brain/.

Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable llm feature circuits. *arXiv*, 2024. doi: 10.48550/arXiv.2406.11944. URL https://arxiv.org/abs/2406.11944.

Karl J. Friston. The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11 (2):127–138, 2010. doi: 10.1038/nrn2787. URL https://doi.org/10.1038/nrn2787.

Yuanjun Gao, Evan Archer, Liam Paninski, and John P Cunningham. Linear dynamical neural population models through nonlinear embeddings. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

Tom M George, Pierre Glaser, Kim Stachenfeld, Caswell Barry, and Claudia Clopath. Simpl: Scalable and hassle-free optimisation of neural representations from behaviour. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

Rabia Gondur, Usama Bin Sikandar, Evan Schaffer, Mikio Christian Aoi, and Stephen L Keeley. Multi-modal gaussian process variational autoencoders for neural and behavioral data. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL http://www.deeplearningbook.org/. Chapter 10: Sequence Modeling—Recurrent and Recursive Nets.

Daniel Hernandez, Antonio Khalil Moretti, Ziqiang Wei, Shreya Saxena, John P Cunningham, and Liam Paninski. Nonlinear evolution via spatially-dependent linear dynamics for electrophysiology and calcium data (vind). *arXiv preprint arXiv:1811.02459*, 2020.

Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of educational psychology*, 24(6):417, 1933.

Patrik O Hoyer. Non-negative matrix factorization with sparseness constraints. *Journal of machine learning research*, 5(May):1457–1469, 2004.

Mark D Humphries. Strong and weak principles of neural dimension reduction. *Neurons, Behavior, Data analysis, and Theory*, 5(2), 2021. ISSN 2690-2664. doi: 10.51628/001c.24619. URL http://dx.doi.org/10.51628/001c.24619.

Cole Hurwitz, Akash Srivastava, Kai Xu, Justin Jude, Matthew G Perich, Lee E Miller, and Matthias H Hennig. Targeted neural dynamical modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Kristopher T Jensen, Ta-Chu Kao, Marco Tripodi, and Guillaume Hennequin. Manifold gplvms for discovering non-euclidean latent structure in neural data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Adam Karvonen, Can Rager, Johnny Lin, Curt Tigges, Joseph Bloom, David Chanin, Yeu-Tong Lau, Eoin Farrell, Callum McDougall, Kola Ayonrinde, Demian Till, Matthew Wearden, Arthur Conmy, Samuel Marks, and Neel Nanda. Saebench: A comprehensive benchmark for sparse autoencoders in language model interpretability. *arXiv*, 2025. URL https://arxiv.org/abs/2503.09532.

Mohammad R Keshtkaran and Chethan Pandarinath. A large-scale neural network training framework for generalized estimation of single-trial population dynamics. *Nature Methods*, 19(12): 1598–1608, 2022.

Timothy Doyeon Kim, Thomas Zhihao Luo, Jonathan W Pillow, and Carlos D Brody. Inferring latent dynamics underlying neural population activity via neural differential equations. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pp. 5568–5578. PMLR, 2021.

Nina Kudryashova, Matthew G Perich, Lee E Miller, and Matthias H Hennig. Ctrl-tndm: Decoding feedback-driven movement corrections from motor cortex neurons. In *Computational and Systems Neuroscience (Cosyne) Abstracts*, 2023.

Trung Le and Eli Shlizerman. Stndt: Modeling neural population activity with spatiotemporal transformers. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 17926–17939. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/72163d1c3c1726f1c29157d06e9e93c1-Paper-Conference.pdf.

Alexander LeNail. Nn-svg: Publication-ready neural network architecture schematics. *Journal of Open Source Software*, 4(33):747, 2019. doi: 10.21105/joss.00747. URL https://doi.org/10.21105/joss.00747.

Jack Lindsey, Adly Templeton, Jonathan Marcus, Thomas Conerly, Joshua Batson, and Christopher Olah. Sparse crosscoders for cross-layer features and model diffing. *Transformer Circuits Thread*, 2024. URL https://transformer-circuits.pub/2024/crosscoders/index.html.

Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/biology.html.

Ryan J Low, Sam Lewallen, Dmitriy Aronov, Rachel Nevers, and David W Tank. Probing variability in a cognitive map using manifold inference from neural dynamics. *bioRxiv*, pp. 418939, 2018.

Jakob H Macke, Lars Buesing, John P Cunningham, Byron M Yu, Krishna V Shenoy, and Maneesh Sahani. Empirical models of spiking in neural populations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, 2011.

Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

Bruno A. Olshausen and David J. Field. Sparse coding with an overcomplete basis set: A strategy employed by V1? *Vision Research*, 37(23):3311–3325, 1997. doi: 10.1016/S0042-6989(97)00169-7. URL https://doi.org/10.1016/S0042-6989(97)00169-7.

Dimitre G Ouzounov, Tianyu Wang, Mengran Wang, Danielle D Feng, Nicholas G Horton, Jean C Cruz-Hernández, Yu-Ting Cheng, Jacob Reimer, Andreas S Tolias, Nozomi Nishimura, and Chris Xu. In vivo three-photon imaging of activity of gcamp6-labeled neurons deep in intact mouse brain. *Nature Methods*, 14(4):388–390, 2017. doi: 10.1038/nmeth.4183. URL https://doi.org/10.1038/nmeth.4183.

Marius Pachitariu, Nicholas Steinmetz, Shabnam Kadir, Matteo Carandini, and Harris Kenneth D. Kilosort: realtime spike-sorting for extracellular electrophysiology with hundreds of channels. *bioRxiv*, 2016. doi: 10.1101/061481. URL https://www.biorxiv.org/content/early/2016/06/30/061481.

Marius Pachitariu, Carsen Stringer, Mario Dipoppa, Sylvia Schröder, L. Federico Rossi, Henry Dalgleish, Matteo Carandini, and Kenneth D. Harris. Suite2p: beyond 10,000 neurons with standard two-photon microscopy. *bioRxiv*, 2017. doi: 10.1101/061507. URL https://www.biorxiv.org/content/early/2017/07/20/061507.

Marten Pals, Vineeth Ramaswamy, Léo Grinsztajn, Maxime Le Helley, and Etienne Roesch. Inferring stochastic low-rank recurrent neural networks with variational sequential monte carlo. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Arthur Pellegrino, Heike Stein, and N. Alex Cayco-Gajic. Dimensionality reduction beyond neural subspaces with slice tensor component analysis. *Nature Neuroscience*, 27(6):1199–1210, 2024.

Sean M Perkins, Elom A Amematsro, John P Cunningham, Qi Wang, and Mark M Churchland. An emerging view of neural geometry in motor cortex supports high-performance decoding. *eLife (Reviewed Preprint, Version of Record)*, 2025. doi: 10.7554/eLife.89421. URL https://elifesciences.org/reviewed-preprints/89421.

Jonathan W. Pillow, Jonathon Shlens, Liam Paninski, Alexander Sher, Alan M. Litke, E. J. Chichilnisky, and Eero P. Simoncelli. Spatio-temporal correlations and visual signalling in a complete neuronal population. *Nature*, 454(7207):995–999, 2008. doi: 10.1038/nature07140. URL https://doi.org/10.1038/nature07140.

Bogdan C. Raducanu, Refet Firat Yazicioglu, Carolina Mora López, Marco Ballini, Jan Putzeys, Shiwei Wang, Andrei Alexandru, Véronique Rochus, Marleen Welkenhuysen, Nick Van Helleputte, Silke Musa, Robert Puers, Fabian Kloosterman, Chris Van Hoof, Richárd Fiáth, István Ulbert, and Srinjoy Mitra. Time multiplexed active neural probe with 1356 parallel recording sites. *Sensors*, 17(10):2388, 2017. doi: 10.3390/s17102388. URL https://doi.org/10.3390/s17102388.

Rajesh P. N. Rao and Dana H. Ballard. Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999. doi: 10.1038/4580. URL https://doi.org/10.1038/4580.

Omid G Sani, Bijan Pesaran, and Maryam M Shanechi. Dissociative and prioritized modeling of behaviorally relevant neural dynamics using recurrent neural networks. *Nature Neuroscience*, 27 (10):2033–2045, 2024.

Mathieu Schimel, Ta-Chu Kao, Kristopher T Jensen, and Guillaume Hennequin. ilqr-vae: Control-based learning of input-driven dynamics with applications to neural data. In *The Tenth International Conference on Learning Representations (ICLR)*, 2022.

Valentin Schmutz, Ali Haydaroğlu, Shuqi Wang, Yixiao Feng, and Kenneth D Harris. High-dimensional neuronal activity from low-dimensional latent dynamics: a solvable model. *bioRxiv*, 2025. URL https://www.biorxiv.org/content/10.1101/2025.06.03.657632v1.

Steffen Schneider, Jin H Lee, and Mackenzie W Mathis. Learnable latent embeddings for joint behavioural and neural analysis. *Nature*, 617(7960):360–368, 2023.

Krishna V. Shenoy, Maneesh Sahani, and Mark M. Churchland. Cortical control of arm movements: A dynamical systems perspective. *Annual Review of Neuroscience*, 36:337–359, 2013. doi: 10.1146/annurev-neuro-062111-150509. URL https://doi.org/10.1146/annurev-neuro-062111-150509.

Joshua H. Siegle, Xiaoxuan Jia, Séverine Durand, Sam Gale, Corbett Bennett, Nile Graddis, Greggory Heller, Tamina K. Ramirez, Hannah Choi, Jennifer A. Luviano, Peter A. Groblewski, Ruweida Ahmed, Anton Arkhipov, Amy Bernard, Yazan N. Billeh, Dillan Brown, Michael A. Buice, Nicolas Cain, Shiella Caldejon, Linzy Casal, Andrew Cho, Maggie Chvilicek, Timothy C. Cox, Kael Dai, Daniel J. Denman, Saskia E. J. de Vries, Roald Dietzman, Luke Esposito, Colin Farrell, David Feng, John Galbraith, Marina Garrett, Emily C. Gelfand, Nicole Hancock, Julie A. Harris, Robert Howard, Brian Hu, Ross Hytnen, Ramakrishnan Iyer, Erika Jessett, Katelyn Johnson, India Kato, Justin Kiggins, Sophie Lambert, Jerome Lecoq, Peter Ledochowitsch, Jung Hoon Lee, Arielle Leon, Yang Li, Elizabeth Liang, Fuhui Long, Kyla Mace, Jose Melchior, Daniel Millman, Tyler Mollenkopf, Chelsea Nayan, Lydia Ng, Kiet Ngo, Thuyahn Nguyen, Philip R. Nicovich, Kat North, Gabriel Koch Ocker, Doug Ollerenshaw, Michael Oliver, Marius Pachitariu, Jed Perkins, Melissa Reding, David Reid, Miranda Robertson, Kara Ronellenfitch, Sam Seid, Cliff Slaughterbeck, Michelle Stoecklin, David Sullivan, Ben Sutton, Jackie Swapp, Carol Thompson, Kristen Turner, Wayne Wakeman, Jennifer D. Whitesell, Derric Williams, Ali Williford, Rob Young, Hongkui Zeng, Sarah Naylor, John W. Phillips, R. Clay Reid, Stefan Mihalas, Shawn R. Olsen, and Christof Koch. A survey of spiking activity reveals a functional hierarchy of mouse corticothalamic visual areas. *Nature*, 592(7876):86–92, 2021.

Yue Song, T. Anderson Keller, Yisong Yue, Pietro Perona, and Max Welling. Langevin flows for modeling neural latent dynamics. 2025. URL https://arxiv.org/abs/2507.11531.

Nicholas A. Steinmetz, Cagatay Aydin, Anna Lebedeva, Michael Okun, Marius Pachitariu, Marius Bauza, Maxime Beau, Jai Bhagat, Claudia Böhm, Martijn Broux, Susu Chen, Jennifer Colonell, Richard J. Gardner, Bill Karsh, Fabian Kloosterman, Dimitar Kostadinov, Carolina Mora-Lopez, John O'Callaghan, Junchol Park, Jan Putzeys, Britton Sauerbrei, Rik J. J. van Daal, Abraham Z.

Vollan, Shiwei Wang, Marleen Welkenhuysen, Zhiwen Ye, Joshua T. Dudman, Barundeb Dutta, Adam W. Hantman, Kenneth D. Harris, Albert K. Lee, Edvard I. Moser, John O'Keefe, Alfonso Renart, Karel Svoboda, Michael Häusser, Sebastian Haesler, Matteo Carandini, and Timothy D. Harris. Neuropixels 2.0: A miniaturized high-density probe for stable, long-term brain recordings. *Science*, 372(6539):eabf4588, 2021. doi: 10.1126/science.abf4588. URL https://doi.org/10.1126/science.abf4588.

David Sussillo and Omri Barak. Opening the black box: Low-dimensional dynamics in high-dimensional recurrent neural networks. *Neural Computation*, 25(3):626–649, 2013. doi: 10.1162/NECO_a_00409. URL https://doi.org/10.1162/NECO_a_00409.

Floris Takens. Detecting strange attractors in turbulence. In David A. Rand and L.-S. Young (eds.), *Dynamical Systems and Turbulence, Warwick 1980*, volume 898 of *Lecture Notes in Mathematics*, pp. 366–381. Springer, 1981. doi: 10.1007/BFb0091924. URL https://doi.org/10.1007/BFb0091924.

Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L. Turner, Callum McDougall, Monte MacDiarmid, Alex Tamkin, Esin Durmus, Tristan Hume, Francesco Mosconi, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 sonnet. *Transformer Circuits Thread*, 2024. URL https://transformer-circuits.pub/2024/scaling-monosemanticity/.

Wilson Truccolo, Uri T. Eden, Matthew R. Fellows, John P. Donoghue, and Emery N. Brown. A point process framework for relating neural spiking activity to spiking history, neural ensemble, and extrinsic covariate effects. *Journal of Neurophysiology*, 93(2):1074–1089, 2005. doi: 10.1152/jn.00697.2004. URL https://doi.org/10.1152/jn.00697.2004.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.

Vincent Villette, Mariya Chavarha, Ivan Dimov, Jonathan Bradley, Lagnajeet Pradhan, Benjamin Mathieu, Stephen W. Evans, Simon Chamberland, Dongqing Shi, Renzhi Yang, Benjamin B. Kim, Annick Ayon, Abdelali Jalil, François St-Pierre, Mark J. Schnitzer, Guo-Qiang Bi, Katalin Tóth, Jun B. Ding, Stéphane Dieudonné, and Michael Z. Lin. Ultrafast two-photon imaging of a high-gain voltage indicator in awake behaving mice. *Cell*, 179(7):1590–1608.e23, 2019. doi: 10.1016/j.cell.2019.11.004. URL https://doi.org/10.1016/j.cell.2019.11.004.

Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pp. 1096–1103, Helsinki, Finland, 2008. ACM. doi: 10.1145/1390156.1390294. URL https://doi.org/10.1145/1390156.1390294.

Joel Ye, Jennifer L Collinger, Leila Wehbe, and Robert Gaunt. Neural data transformer 2: Multi-context pretraining for neural spiking activity. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Byron M Yu, John P Cunningham, Gokhan Santhanam, Stephen I Ryu, Krishna V Shenoy, and Maneesh Sahani. Gaussian-process factor analysis for low-dimensional single-trial analysis of population spike trains. *Journal of Neurophysiology*, 102(1):614–635, 2009.

Yizi Zhang, Yanchen Wang, Donato Jimenez-Beneto, Zixuan Wang, Mehdi Azabou, Blake Richards, Olivier Winter, International Brain Laboratory, Eva Dyer, Liam Paninski, and Cole Hurwitz. Towards a "universal translator" for neural dynamics at single-cell, single-spike resolution. 2024. URL https://arxiv.org/abs/2407.14668.

ACKNOWLEDGEMENTS

To maintain blinding, acknowledgements in this submission are omitted, and will be included in the final version.

5 APPENDIX

5.1 SOFTWARE AND DATA AVAILABILITY

To maintain blinding, software and data availability information can be found in the anonymized repository at . . . .

5.2 ON INTERPRETABLE LATENTS

Many neural LVM approaches focus on learning structure in a low-dimensional latent space. However, this structure can be inherently complex and difficult to interpret. NLDisco bypasses this challenge by learning individual latents directly.

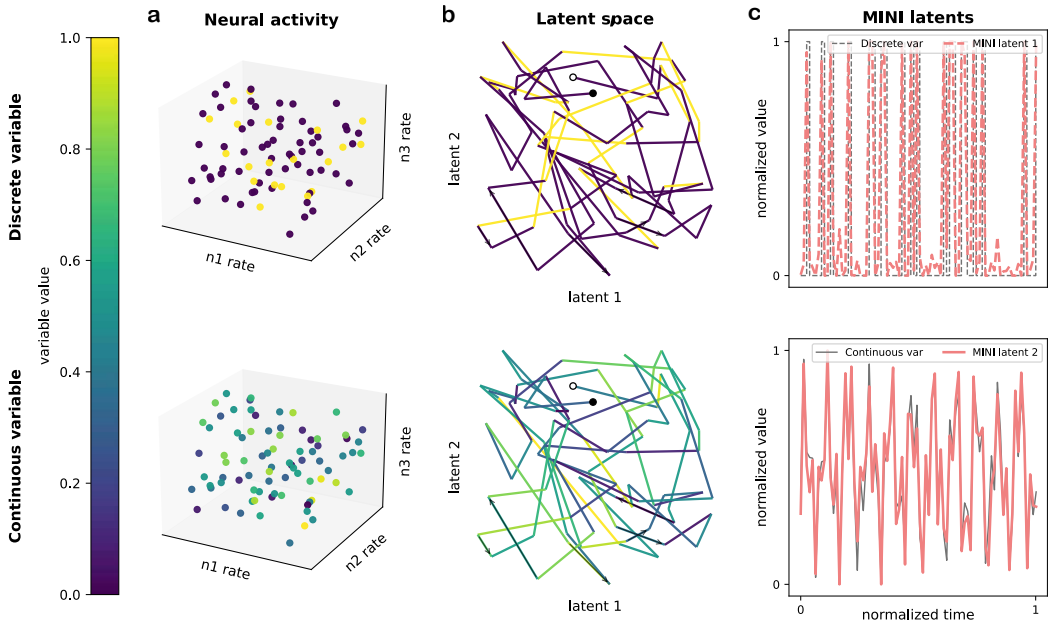

Figure S1: Interpretable latents in a complex latent space.
This toy example highlights the utility NLDisco. The two rows show two different variables (top: discrete; bottom: continuous), each uniquely encoded by the same underlying neural activity made up of three neurons' firing rates. The colorbar shows the variables' values as a function of this neural activity. (**a**) Each point in the scatterplots represents a moment in time. (**b**) A projection of this activity into a 2D latent space creates tangled trajectories where variable states (e.g. 'on' and 'off' in the discrete case, and 'high' and 'low' in the continuous case) are not easily distnguished. The start and end points of the trajectories are marked by white and black dots, respectively, while arrows indicate trajectory direction. (**c**) In contrast to the tangled latent space, NLDisco finds individual latents corresponding to each variable, demonstrating the potential for improved interpretability of neural representations.

5.3 ADDITIONAL PIPELINE DETAILS

5.3.1 MODEL TRAINING

Some notes on Model training procedure:

Dead Latent Resurrection. A common failure mode in training SEDs is "latent death," where dictionary elements cease to activate for any input. We address this with an auxiliary loss designed to revive dead latents. We monitor feature activation frequencies and identify a set of dead latents $\mathcal{D}$. These latents are then trained via an auxiliary MSE loss to reconstruct the residual error of the primary model ($\mathbf{x}_{\text{target}} - \hat{\mathbf{x}}_L$). The gradients from this auxiliary loss are exclusively applied to the parameters of the dead latents, thereby encouraging them to learn useful representations without disrupting the training of active latents.

Algorithm 1 Model training procedure

Data definitions:
  Input neural data: $\mathbf{Y} \in \mathbb{R}^{B \times S \times N_{\text{in}}}$ (Batch, Sequence length, Input neural units)
  Target neural data: $\mathbf{Z} \in \mathbb{R}^{B \times \times N_{\text{out}}}$ (Batch, Output neural units)
Model definitions:
  Encoder: $\mathbf{W}_{\text{enc}} \in \mathbb{R}^{D_{\max} \times S \times N_{\text{in}}}$, $\mathbf{b}_{\text{enc}} \in \mathbb{R}^{D_{\max}}$ (Hidden layer neurons)
  Decoder: $\mathbf{W}_{\text{dec}} \in \mathbb{R}^{N_{\text{out}} \times D_{\max}}$, $\mathbf{b}_{\text{dec}} \in \mathbb{R}^{N_{\text{out}}}$
  Transformer block (optional) $\theta_{\mathbf{t}}$: $\{\mathbf{W}_{Q,K,V} \in \mathbb{R}^{D_{\max} \times D_{\max}}, \dots\}$
  Matryoshka levels: $\{\mathbf{D_l}\}_{\mathbf{l=1}}^{\mathbf{L}}$
  Weights each level's reconstruction loss (optional): $\{\lambda_{\mathbf{l}}\}_{\mathbf{l=1}}^{\mathbf{L}}$
  Auxiliary loss weight: γ

procedure TRAIN_STEP($\mathbf{Y}, \mathbf{Z}$)

▷ — *Forward Pass* —
  $\mathbf{A} \leftarrow \text{ReLU}(\mathbf{Y}\mathbf{W}_{\text{enc}}^T + \mathbf{b}_{\text{enc}})$ ▷ Get encoder activations

  if $S > 1$ **then**
    $\mathbf{A} \leftarrow \text{SelfAttention}(\mathbf{A}; \theta_{\text{attn}})$ ▷ Temporally integrate over latent space
  end if

  $\mathcal{L}_{\text{recon}} \leftarrow 0$
  for $l = 1$ to L **do** ▷ For each level...
    $\hat{\mathbf{A}}_l \leftarrow S_{\text{topk}}(\mathbf{A}_{:,:D_l})$ ▷ Sparsify latents with batch top-k
    $\hat{\mathbf{Z}}_l \leftarrow \text{ReLU}(\hat{\mathbf{A}}_l \mathbf{W}_{\text{dec},:,:D_l}^T + \mathbf{b}_{\text{dec}})$ ▷ Reconstruct target (decoder activations)
    $\mathcal{L}_{\text{recon}} \leftarrow \mathcal{L}_{\text{recon}} + \lambda_l \cdot \text{MSLE}(\mathbf{Z}, \hat{\mathbf{Z}}_l)$ ▷ Get reconstruction loss
  end for

▷ — *Auxiliary Loss for Dead Latent Resurrection* —
  $\mathcal{D} \leftarrow \text{GetDeadLatents}(\mathbf{A})$
  if any($\mathcal{D}$) **then** ▷ Only compute if dead latents exist
    $\mathbf{R} \leftarrow \mathbf{Z} - \hat{\mathbf{Z}}_L$ ▷ Get reconstruction residual
    $\hat{\mathbf{R}} \leftarrow \text{ReLU}((\mathbf{A} \odot \mathcal{D})\mathbf{W}_{\text{dec}}^T + \mathbf{b}_{\text{dec}})$ ▷ Reconstruct residual from dead latents
    $\mathcal{L}_{\text{aux}} \leftarrow \text{MSE}(\mathbf{R}, \hat{\mathbf{R}})$ ▷ Get auxiliary loss
  else
    $\mathcal{L}_{\text{aux}} \leftarrow 0$
  end if

  $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{recon}} + \gamma \mathcal{L}_{\text{aux}}$ ▷ Total loss: reconstruction + auxiliary

▷ — *Backward Pass & Parameter Update* —
  $\mathbf{g} \leftarrow \nabla_\theta \mathcal{L}_{\text{total}}$ ▷ Compute gradients, masking $\nabla \mathcal{L}_{\text{aux}}$ to dead latents
  $\theta \leftarrow \text{Update}(\theta, \mathbf{g})$ ▷ Update model parameters
end procedure

5.3.2 MODEL ARCHITECTURE

The core of NLDisco is the implementation of overcomplete sparse encoder-decoder models with single hidden layers and ReLU activations. While we experimented with more complex model architectures, such as multi-layer decoders, we found that they often made interpreting the resulting latents more difficult without significant reconstruction gains. The Matryoshka architecture is particularly useful for datasets where features are expected to exist at multiple scales, as it helps mitigate "feature absorption" where general features are partially subsumed by more specific ones (TODO see Allen results with and without Matryoshka architecture). And the transformer block is most beneficial when analyzing data with meaningful temporal dynamics, as it allows the model to learn features that evolve over a sequence rather than just instantaneous neural patterns (TODO see Aeon results with and without transformer block).

5.3.3 MODEL EVALUATION

Model evaluation centers on the trade-off between reconstruction fidelity and latent dictionary sparsity. High reconstruction fidelity is trivial to achieve with a large and dense code but undermines the goal of finding disentangled, interpretable latents. NLDisco therefore first reviews dictionary quality via latent sparsity metrics (Figure S2 top). The mean L0 norm measures the average number of active latents per sample, set indirectly by the batch top-k parameter, while the latent activity density visualizes the firing distribution across all latents. Together these metrics reveal whether the model has learned an efficient code that uses many latents intermittently while avoiding common pitfalls such as "representational collapse" into too many constantly active latents and/or widespread "feature death", in which a large portion of the dictionary never activates. These sparsity metrics are complemented by a couple of reconstruction fidelity metrics: variance explained (R^2) and cosine similarity of reconstruction-to-target neural activity across both spatial and temporal dimensions (Figure S2 bottom). High scores in these metrics confirm that the sparse code is a sufficient representation that has captured salient information present in the target neural data.

For a deeper diagnosis, several additional optional metrics are available. To check whether explanatory power is well-distributed, a variance attribution analysis quantifies the overall reconstruction variance explained by each individual latent (TODO). And to check if temporal information present in the target neural data is preserved by the latent reconstruction, a spectral frequency analysis compares the reconstruction's frequency content to that of the target neural data, given a specified duration (TODO).

5.3.4 LATENTS EVALUATION

We recommend building simple, bespoke dashboards for viewing the activity of model latents in the context of relevant environmental and/or behavioral variables.

(TODO: show dashboards for Churchland, Allen, and Aeon datasets).

5.3.5 HYPERPARAMETER SWEEPS

We typically sweep over key architectural parameters, including the number and size of Matryoshka levels, the batch top-k sparsity coefficient, and the latent space sequence length for the self-attention layer. For the optimizer, we use Adam by default due to its robust performance across a wide range of tasks. We found popular variants to offer negligible benefits for our use case: direct sparsity enforcement obviates the need for weight decay (AdamW), our shallow models do not require layer-wise adaptive optimizers (LAdam), and additional tuning overhead for Nesterov momentum (NAdam) did not yield a corresponding performance gain.

While hyperparameter sweeps are an empirical science, we typically choose these heuristics for setting sweep ranges...

5.4 ADDITIONAL RESULTS

5.4.1 SIMULATED RAT SPIKE DATA IN A NAVIGATION TASK

Dataset info

- Exp metadata & info
  - Session duration
  - Stimulus conditions
- Neural data info
  - Brain regions
  - Number of units
  - Number of spikes
  - Spike summary stats
- Recreation instructions

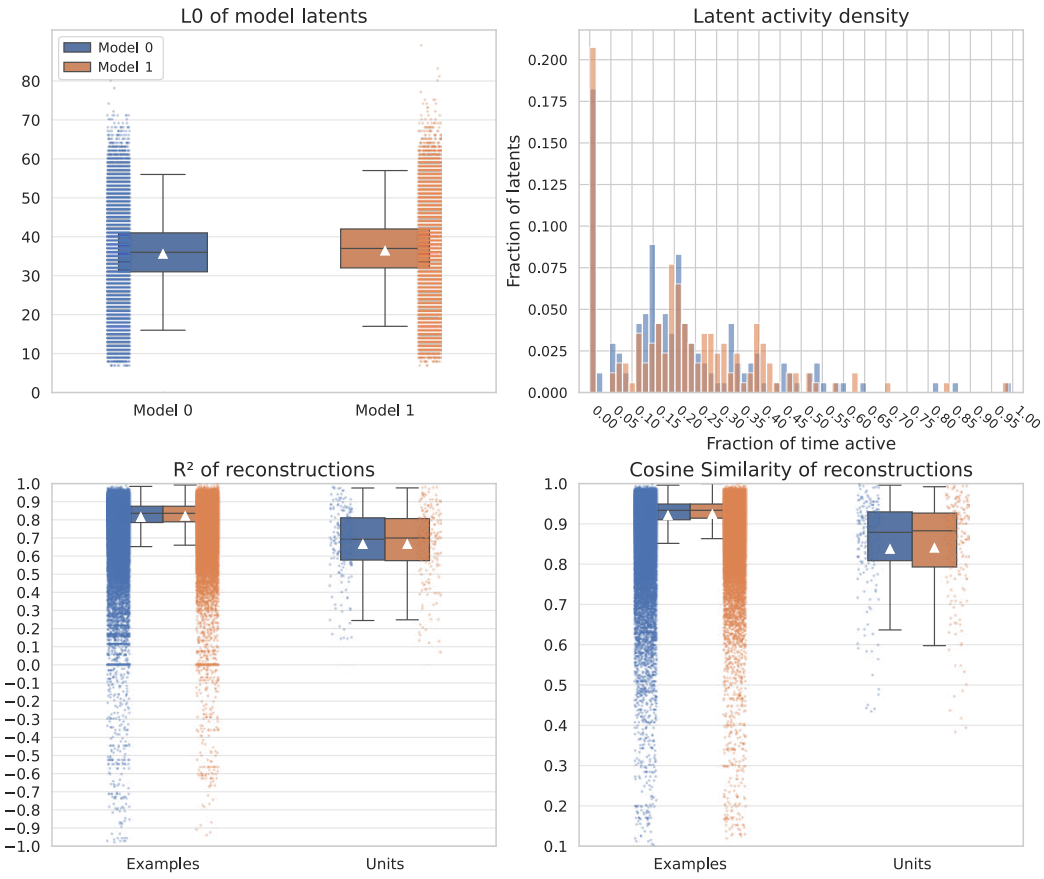

Figure S2: Model evaluation metrics.
This example SAE model evaluation examines a three-level Matryoshka architecture with a total of 320 latents and top-k selections of $12 \cdot B$, $24 \cdot B$, and $36 \cdot B$ active latents per batch size B (1024). The recording had a total of 200 units, and the model was trained on 182,868 samples of 50 ms binned spike counts. The model exhibits an approximately normal distribution for the mean latent L0 norm (TODO metric), few dead or tonically active latents, and high neural reconstruction fidelity as measured by the R^2 and cosine similarity metrics across both units and examples, all indicators of healthy training.

Latent comparisons with sparseNMF, t-SNE, and UMAP

5.4.2 Real macaque spike data in a reaching task

Dataset info

- Exp metadata & info
  - Session duration
  - Stimulus conditions
- Neural data info
  - Brain regions
  - Number of units
  - Number of spikes
  - Spike summary stats
- Access instructions

Latent comparisons with sparseNMF, t-SNE, and UMAP

5.4.3 REAL MOUSE SPIKE DATA IN AN ETHOLOGICAL FORAGING TASK

Dataset info

- Exp metadata & info
  - Session duration
  - Stimulus conditions
- Neural data info
  - Brain regions
  - Number of units
  - Number of spikes
  - Spike summary stats
- Access instructions

Latent comparisons with sparseNMF, t-SNE, and UMAP

5.4.4 MOUSE SPIKE DATA IN A PASSIVE VISUAL TASK

In addition to the three main results we showcase in the main text (section 3), here we show NLDisco applied to a visual coding Neuropixels dataset (Siegle et al., 2021) from the Allen institute.

Dataset info

- Exp metadata & info
  - Session duration
  - Stimulus conditions
- Neural data info
  - Brain regions
  - Number of units
  - Number of spikes
  - Spike summary stats
- Recreation instructions

As done in section 3, here we also compare a trained NLDisco model to Cebra and LangevinFlow models...

Latent comparisons with sparseNMF, t-SNE, and UMAP Stimulus decoding accuracy:

- Natural image classification: 87.3% (vs. 76.2% for raw data, 81.4% for PCA)
- Motion direction decoding: 94.1% (vs. 88.7% for raw data, 90.2% for ICA)
- Orientation discrimination: 91.6% (vs. 85.3% for raw data, 87.9% for standard SAE)

Behavioral state decoding:

- Locomotion state classification: 88.7% accuracy
- Pupil size (arousal) prediction: Pearson r = 0.73
- Visual attention state: 82.4% accuracy

5.5 METHODS COMPARISON

Here we highlight 15 features of neural LVM methods and create a table displaying how NLDisco and other relevant methods compare against the following features:

- *Requires multimodal data*: Whether the method requires multimodal data (e.g. video data, or various forms of behavioral data, in addition to neural data).
- *Requires trial-structured data*: Whether the method requires trial-structured data.
- *Supports multimodal data*: Whether the method can incorporate multimodal data, even if not required.
- *Supports trial-structured data*: Whether the method can use trial-structured data, even if not required.
- *Learns sparse, easily identifiable latents*: Whether the method by default learns sparse, easily identifiable latents.
- *Learns hierarchical latents*: Whether the method by default learns latents that are hierarchically organized (e.g. whether the method can learn one latent that corresponds to a particular behavior, and another, sparser latent that corresponds to a sub-behavior of the first)
- *Learns temporally precise latents*: Whether the method by default learns latents that are time-locked to neural events at the resolution of the desired event, potentially down to single-spike precision.
- *Learns a continuously-valued latent space*: Whether the method by default learns latents that change smoothly as a function of changes in the input neural data.
- *Can use temporal dynamics to update the latent space*: Whether the method can use temporal dynamics (e.g. neural data or latent space history) to update the latent space.
- *Imposes a prior on the latent space*: Whether the method imposes a predfined structure on the latent space (e.g. geometric constraints like orthogonality of latents, or distributional assumptions like a Gaussian latents).
- *Uses nonlinear dynamics*: Whether the method learns latents from nonlinear neural dynamics.
- *Can be used as a generative model*: Whether the method can be used to generate new neural data samples from the learned latent space.
- *Enforces neural data reconstruction*: Whether the method needs to perform neural data reconstruction when learning latents.
- *Has approximate linear time scaling*: Whether the method has linear time scaling with respect to the number of data points in an example and in the dataset.
- *Requires significant hyperparameter tuning*: Whether the method requires significant hyperparameter tuning to learn interpretable latents.

Light-green text indicates that the method has an ideal implementation of the feature, while dark-red text indicates a shortcoming.

Feature	**NLDisco***	LangevinFlow* (Song et al., 2025)	CEBRA* (Schneider et al., 2023)	ST-NDT (Le & Shlizerman, 2022)	AutoLFADS (Keshtkaran & Pandarinath, 2022)	UMAP (McInnes et al., 2018)	t-SNE (van der Maaten & Hinton, 2008)	sparseNMF** (Hoyer, 2004)	ICA (Comon, 1994)	PCA (Hotelling, 1933)
Requires multimodal data	No	No	No	No	No	No	No	No	No	No
Requires trial-structured data	No	No	No	No	No^	No	No	No	No	No
Supports multimodal data	No†	No	Yes	No	No	No	No	No	No	No
Supports trial-structured data	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Learns sparse, easily identifiable latents	Yes	No	No	No	No	No	No	Yes	No	No
Learns hierarchical latents	Yes	No	No	No	No	No	No	No	No	No
Learns temporally precise latents	Yes	Yes	Yes	Yes	Yes	No	No	Yes	No	No
Learns a continuously-valued latent space	No‡	Yes	Yes	Yes	Yes	Yes	Yes	No‡	Yes	Yes
Can use temporal dynamics to update the latent space	Yes	Yes	Yes	Yes	Yes	No	No	No	No	No
Imposes a prior on the latent space	No	Yes	No	No	Yes	No	No	No	Yes	Yes
Uses nonlinear dynamics	Yes	Yes	Yes	Yes	Yes	No	No	No	No	No
Can be used as a generative model	Yes	Yes	Yes	Yes	Yes	No	No	No	No	No
Enforces neural data reconstruction	Yes	Yes	No	Yes	Yes	No	No	Yes	No	No
Has approximate linear time scaling	Yes#	No	Yes	No	No	Yes	No	No	Yes	Yes
Requires significant hyperparameter tuning	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No

Table 1: Comparison of NLDisco with other neural LVM methods.

*: Latent comparison visualized in Results
**: Latent comparison visualized in Additional results
†: Not a fundamental limitation – could be added as a feature
‡: Enforced sparsity can cause step-like jumps in the data-to-latents mapping
#: In the standard implementation, without a transformer layer in the decoder
^: Can bin data to create "pseudo"-trials

Below we provide a brief justification for each table entry.

- **Requires multimodal data**
  - *PCA* No. Classic PCA factorizes a single data matrix and does not require paired behavioral or other modalities.
  - *sparseNMF* No. NMF with sparsity constraints operates on one non-negative matrix; no second modality is needed.
  - *LangevinFlow* No. It's a sequential VAE for neural population dynamics; it models spikes/rates without needing aligned behavioral inputs.

  - *CEBRA* No. CEBRA supports supervised multimodal training, but it also has an unsupervised "label-free" mode on neural-only data.
  - *ST-NDT* No. The model is trained on spike counts/rates and does not intrinsically require a second modality.
  - *AutoLFADS* No. AutoLFADS infers latent dynamics from neural data alone (behavior is often used only for downstream decoding).
- **Requires trial-structured data**
  - *PCA* No. Matrix decomposition does not assume trials.
  - *sparseNMF* No. Standard NMF does not require trial segmentation.
  - *LangevinFlow* No. It is defined on continuous time series and can be trained on long, unsegmented recordings.
  - *CEBRA* No. CEBRA works across sessions and continuous recordings; its sampling scheme does not require explicit trial boundaries.
  - *ST-NDT* No. Although many datasets are organized as trials, ST-NDT's masked-modeling objective applies to continuous sequences as well.
  - *AutoLFADS* No (footnote in your table). AutoLFADS has been used on continuous data and can be applied without true trial structure (e.g., via windowing/pseudo-trials).
- **Supports multimodal data**
  - *PCA* No. While you can concatenate features, PCA is not designed for cross-modal alignment or contrastive objectives.
  - *sparseNMF* No. The basic objective reconstructs a single non-negative matrix, not joint modalities.
  - *LangevinFlow* No. The described model targets neural dynamics; multimodal coupling is not a core feature.
  - *CEBRA* Yes. It explicitly supports joint neuralbehavioral training (supervised) as well as label-free training, enabling cross-modal embeddings.
  - *ST-NDT* No. It models neural population activity; multimodal alignment is not part of the method.
  - *AutoLFADS* No. The VAE reconstructs neural observations; other modalities are typically used only for evaluation/decoding.
- **Supports trial-structured data**
  - *PCA / sparseNMF / LangevinFlow / CEBRA / ST-NDT / AutoLFADS* Yes. All can be applied to trialized datasets (matrix rows/time bins per trial; masking across trial windows; or VAE sequences per trial).
- **Learns sparse, easily identifiable latents**
  - *PCA* No. Principal components are dense linear combinations and typically not parts-based or sparse without additional penalties.
  - *sparseNMF* Yes. The objective adds explicit sparsity constraints, producing parts-based, interpretable components and sparse activations.
  - *LangevinFlow* No. The sequential VAE uses continuous latent dynamics without an inherent sparsity/parts constraint.
  - *CEBRA* No. The contrastive embedding is learned by an encoder; there's no explicit sparsity/parts prior on latent coordinates.
  - *ST-NDT* No. Transformer representations/rate outputs are not designed to be sparse components.
  - *AutoLFADS* No. LFADS/AutoLFADS learn dense low-dimensional factors unless additional sparsity regularizers are added.
- **Learns hierarchical latents**
  - *PCA / sparseNMF / LangevinFlow / CEBRA / ST-NDT / AutoLFADS* No. None of these methods impose a hierarchical/nesting constraint over features by default. (See their objectives/architectures; no nested-feature prior is specified.)
- **Learns temporally precise latents**

  - *PCA* No. Components are static projections with no temporal state update.
  - *sparseNMF* Yes (per your table). When applied to time-binned data (or with sequence/convolutional extensions), NMF can isolate time-localized motifs, yielding temporally sharp activations even without a dynamics model.
  - *LangevinFlow* Yes. Latents evolve via an underdamped Langevin SDE inside a sequential VAE, giving fine-grained temporal trajectories.
  - *CEBRA* Yes. Its time-aware sampling scheme shapes embeddings to be consistent across nearby time points, supporting temporally resolved latents/decoding.
  - *ST-NDT* Yes. It infers single-trial firing rates at each time bin using spatiotemporal attention and masked reconstruction.
  - *AutoLFADS* Yes. AutoLFADS yields high-time-resolution single-trial rate estimates.
- **Learns a continuously-valued latent space**
  - *PCA* Yes. Continuous linear coordinates (principal components).
  - *sparseNMF* No (per your table). In practice, sparsity constraints drive near-binary/parts-activation patterns rather than a smooth, unconstrained continuous code.
  - *LangevinFlow* Yes. Latents follow continuous SDE dynamics.
  - *CEBRA* Yes. Encoded embeddings are continuous vectors optimized by a contrastive loss.
  - *ST-NDT* Yes. The model predicts continuous firing-rate trajectories (with spikes modeled as Poisson draws from those rates).
  - *AutoLFADS* Yes. LFADS/AutoLFADS learn continuous latent factors/generator states.
- **Can use temporal dynamics to update the latent space**
  - *PCA* No. No state evolution; projections are static.
  - *sparseNMF* No. Standard NMF has no transition prior/dynamics.
  - *LangevinFlow* Yes. Latents evolve via Langevin dynamics between time steps.
  - *CEBRA* Yes. The training pairs/sampling function incorporate temporal neighborhoods, effectively using time to shape the embedding.
  - *ST-NDT* Yes. Self-attention across time learns temporal dependencies and updates inferred rates accordingly.
  - *AutoLFADS* Yes. The generator RNN (and inferred inputs) explicitly model latent temporal evolution.
- **Imposes a prior on the latent space**
  - *PCA* Yes. In the PPCA view, latents have an isotropic Gaussian prior; PCA emerges as ML under this generative model.
  - *sparseNMF* No. It is an optimization with non-negativity/sparsity constraints, not a probabilistic latent prior.
  - *LangevinFlow* Yes. The latent time evolution is governed by a physical prior (underdamped Langevin SDE/potential), combined with a VAE likelihood.
  - *CEBRA* No. It uses a contrastive objective; there's no explicit generative prior over latents.
  - *ST-NDT* No. Training is via masked reconstruction/contrastive loss, not a latent prior.
  - *AutoLFADS* Yes. As a VAE, it places priors over latent initial conditions/inputs/dynamics.
- **Uses nonlinear dynamics**
  - *PCA* No. Linear projection model.
  - *sparseNMF* No. Linear parts-based factorization.
  - *LangevinFlow* Yes. Nonlinear latent dynamics via learned potential and stochastic forces.
  - *CEBRA* Yes. Nonlinear encoder network learns the embedding.
  - *ST-NDT* Yes. Transformer attention is a nonlinear mapping.
  - *AutoLFADS* Yes. Generator RNN defines nonlinear latent dynamics.

- **Can be used as a generative model**
  - *PCA* No. PCA is not a generative model (absent the PPCA extension's explicit likelihood).
  - *sparseNMF* No. Standard NMF reconstructs the data matrix but is not typically used for sampling new sequences.
  - *LangevinFlow* Yes. It is a VAE with Langevin latent dynamics, enabling simulation/decoding from the learned generative process.
  - *CEBRA* No. The authors emphasize that CEBRA is not a generative model.
  - *ST-NDT* Yes. Given inferred rate trajectories and a Poisson observation model, one can generate spikes by sampling from the inhomogeneous Poisson with the model's rates.
  - *AutoLFADS* Yes. LFADS/AutoLFADS are VAEs that generate neural activity via a latent dynamical system and observation model.
- **Enforces neural data reconstruction**
  - *PCA* Yes. Minimizes reconstruction error under a rank constraint (equivalently maximizes variance explained).
  - *sparseNMF* Yes. Objective reconstructs the input with non-negativity and sparsity penalties.
  - *LangevinFlow* Yes. Trained via an ELBO with a likelihood term that reconstructs neural observations from latents.
  - *CEBRA* No. It optimizes a contrastive loss rather than a reconstruction loss.
  - *ST-NDT* Yes. Uses masked modeling (reconstructing held-out bins) and an auxiliary contrastive loss.
  - *AutoLFADS* Yes. As a VAE, it reconstructs spikes/rates through the decoder likelihood.
- **Has approximate linear time scaling**
  - *PCA* Yes. With randomized/incremental PCA, computation scales near-linearly with the number of samples.
  - *sparseNMF* No. Iterative multiplicative/gradient updates with sparsity constraints can be comparatively slow and require many passes/hyperparameters.
  - *LangevinFlow* Yes. Mini-batch VAE training scales roughly linearly in dataset size. (Model is designed for large neural recordings.)
  - *CEBRA* Yes. Contrastive encoders trained with mini-batches scale well to large, multi-session datasets.
  - *ST-NDT* No. Spatiotemporal self-attention is quadratic in sequence length (and also attends across neurons), so compute grows super-linearly.
  - *AutoLFADS* No. Although each SGD step is linear in batch size, the heavy hyperparameter search (PBT) makes effective compute scale poorly.
- **Requires significant hyperparameter tuning**
  - *PCA* No. Essentially parameter-free aside from the chosen rank.
  - *sparseNMF* Yes. You must pick rank and sparsity levels; performance depends strongly on these choices.
  - *LangevinFlow* Yes. Sequential VAE with SDE dynamics introduces choices for latent dimensionality, dynamics hyperparameters, and optimizer settings.
  - *CEBRA* Yes. Results depend on encoder architecture, temperature, sampling function, supervision mode, etc.
  - *ST-NDT* Yes. Transformer depth/width, attention configuration, masking schedule, and losses require tuning.
  - *AutoLFADS* Yes. The method is explicitly built around automated hyperparameter search (population-based training).